Operating System
Unit – I

Dr. U. Padmavathi

Assistant Professor

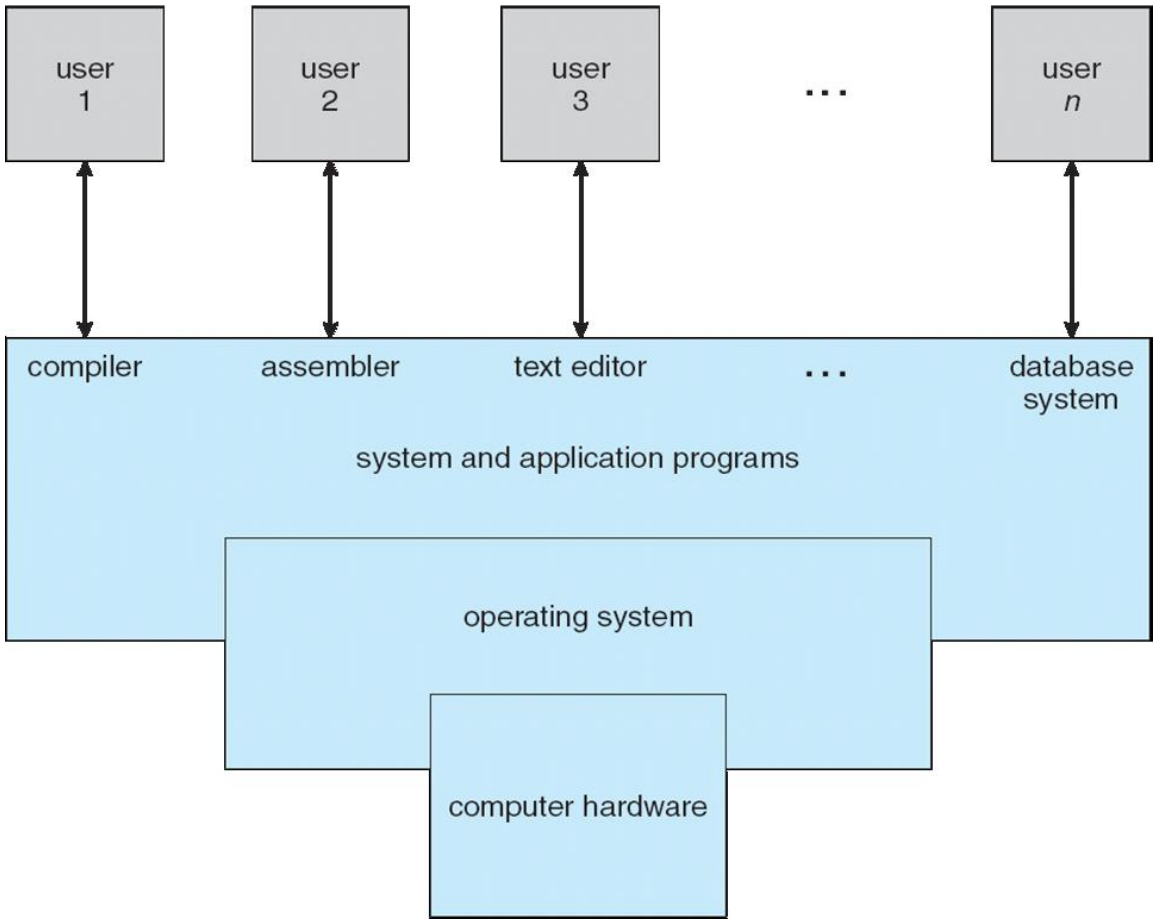
Shiv Nadar University Chennai

What is an Operating System?

- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:
  - Execute user programs and make solving user problems easier
  - Make the computer system convenient to use
  - Use the computer hardware in an efficient manner

Computer System Structure

- Computer system can be divided into four components:
  - Hardware – provides basic computing resources
    - CPU, memory, I/O devices
  - Operating system
    - Controls and coordinates use of hardware among various applications and users
  - Application programs – define the ways in which the system resources are used to solve the computing problems of the users
    - Word processors, compilers, web browsers, database systems, video games
  - Users
    - People, machines, other computers


user
1
user
2
user
3
...
user
n
compiler
assembler
text editor
...
database
system
system and application programs
operating system
computer hardware

What operating system do?

- Depends on the point of view
- Users want convenience, **ease of use** and **good performance**
  - Don't care about **resource utilization**
- But shared computer such as **mainframe** or **minicomputer** must keep all users happy
- Users of dedicate systems such as **workstations** have dedicated resources but frequently use shared resources from **servers**
- Handheld computers are resource poor, optimized for usability and battery life
- Some computers have little or no user interface, such as embedded computers in devices and automobiles

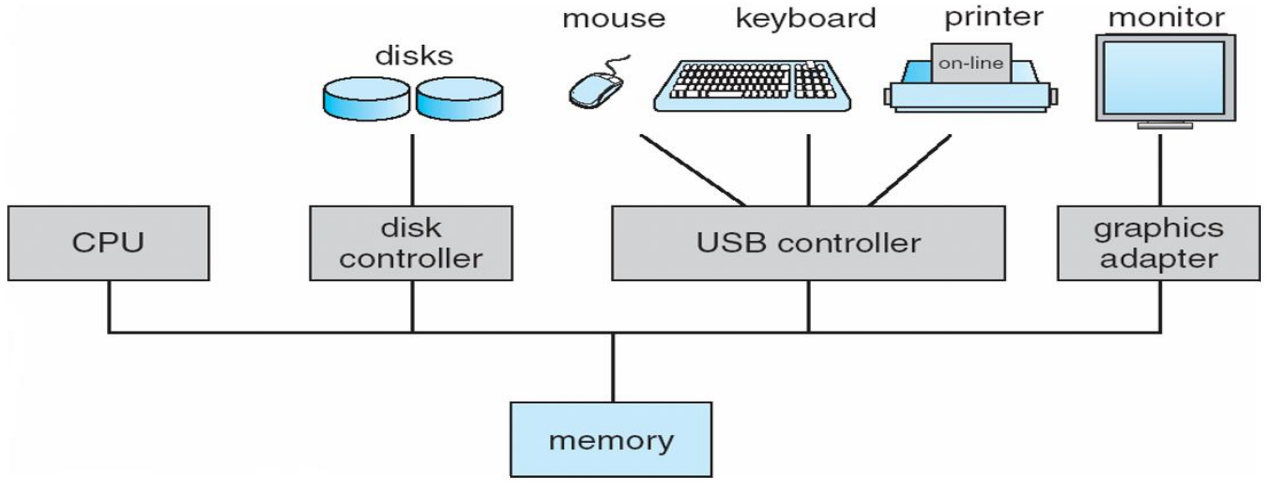
- OS is a **resource allocator**
  - Manages all resources
  - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
  - Controls execution of programs to prevent errors and improper use of the computer

Defining Operating System

- No universally accepted definition
- “Everything a vendor ships when you order an operating system” is a good approximation
  - But varies wildly
- “The one program running at all times on the computer” is the **kernel**.
- Everything else is either
  - a system program (ships with the operating system) , or
  - an application program.

Computer System Organization

- Computer-system operation
  - One or more CPUs, device controllers connect through common bus providing access to shared memory
  - Concurrent execution of CPUs and devices competing for memory cycles



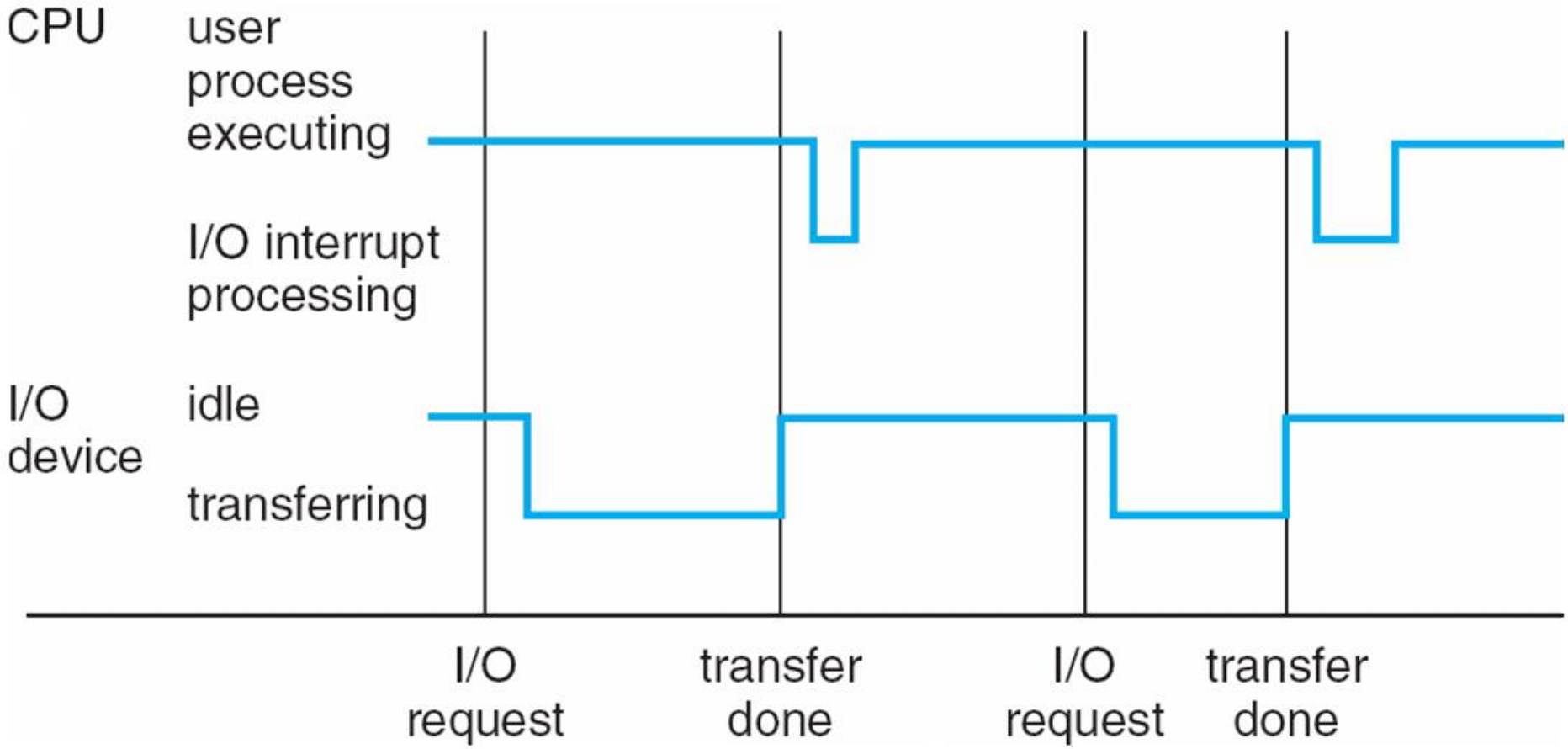
Computer-System Operation

- I/O devices and the CPU can execute concurrently
- Each device controller is in charge of a particular device type
- Each device controller has a local buffer
- CPU moves data from/to main memory to/from local buffers
- I/O is from the device to local buffer of controller
- Device controller informs CPU that it has finished its operation by causing an interrupt

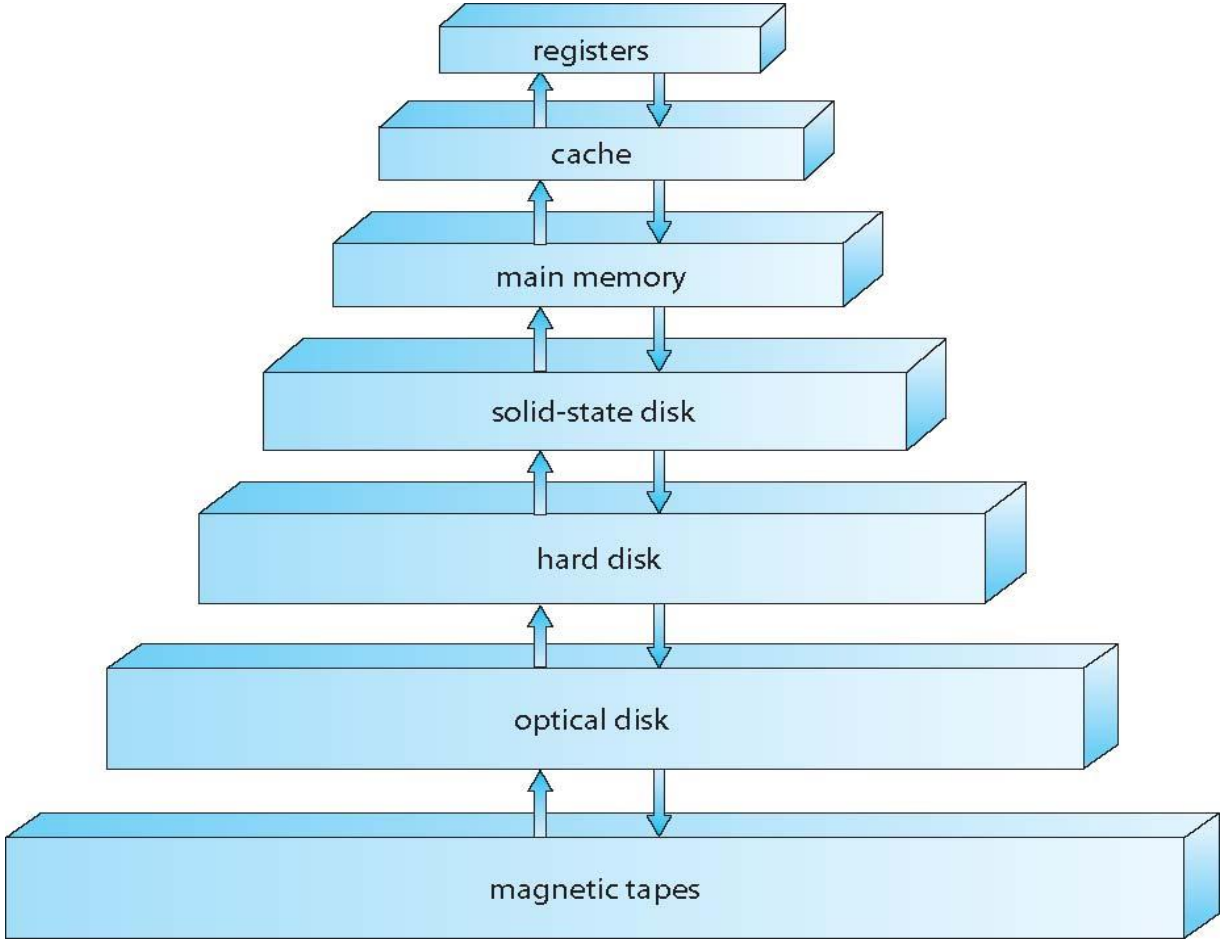
Common Functions of Interrupts

- Interrupt transfers control to the interrupt service routine generally, through the **interrupt vector**, which contains the addresses of all the service routines
- Interrupt architecture must save the address of the interrupted instruction
- A **trap** or **exception** is a software-generated interrupt caused either by an error or a user request
- An operating system is **interrupt driven**

Interrupt Timeline



Storage-Device Hierarchy



Storage Structure

- Main memory – only large storage media that the CPU can access directly
  - **Random access**
  - Typically **volatile**
- Secondary storage – extension of main memory that provides large **nonvolatile** storage capacity
- Hard disks – rigid metal or glass platters covered with magnetic recording material
  - Disk surface is logically divided into **tracks**, which are subdivided into **sectors**
  - The **disk controller** determines the logical interaction between the device and the computer
- **Solid-state disks** – faster than hard disks, nonvolatile
  - Various technologies
  - Becoming more popular

Storage Definitions and Notation Review

- The basic unit of computer storage is the **bit**. A bit can contain one of two values, 0 and 1. All other storage in a computer is based on collections of bits. Given enough bits, it is amazing how many things a computer can represent: numbers, letters, images, movies, sounds, documents, and programs, to name a few. A 8 bits, and on most computers it is the smallest convenient chunk of storage**byte** is . For example, most computers don't have an instruction to move a bit but do have one to move a byte. A less common term is **word**, which is a given computer architecture's native unit of data. A word is made up of one or more bytes. For example, a computer that has 64-bit registers and 64-bit memory addressing typically has 64-bit (8-byte) words. A computer executes many operations in its native word size rather than a byte at a time.

- Computer storage, along with most computer throughput, is generally measured and manipulated in bytes and collections of bytes.
- A **kilobyte**, or **KB**, is 1,024 bytes
- a **megabyte**, or **MB**, is $1{,}024^2$ bytes
- a **gigabyte**, or **GB**, is $1{,}024^3$ bytes
- a **terabyte**, or **TB**, is $1{,}024^4$ bytes
- a **petabyte**, or **PB**, is $1{,}024^5$ bytes

- Computer manufacturers often round off these numbers and say that a megabyte is 1 million bytes and a gigabyte is 1 billion bytes. Networking measurements are an exception to this general rule; they are given in bits (because networks move data a bit at a time).

Interaction

- Load
- Store

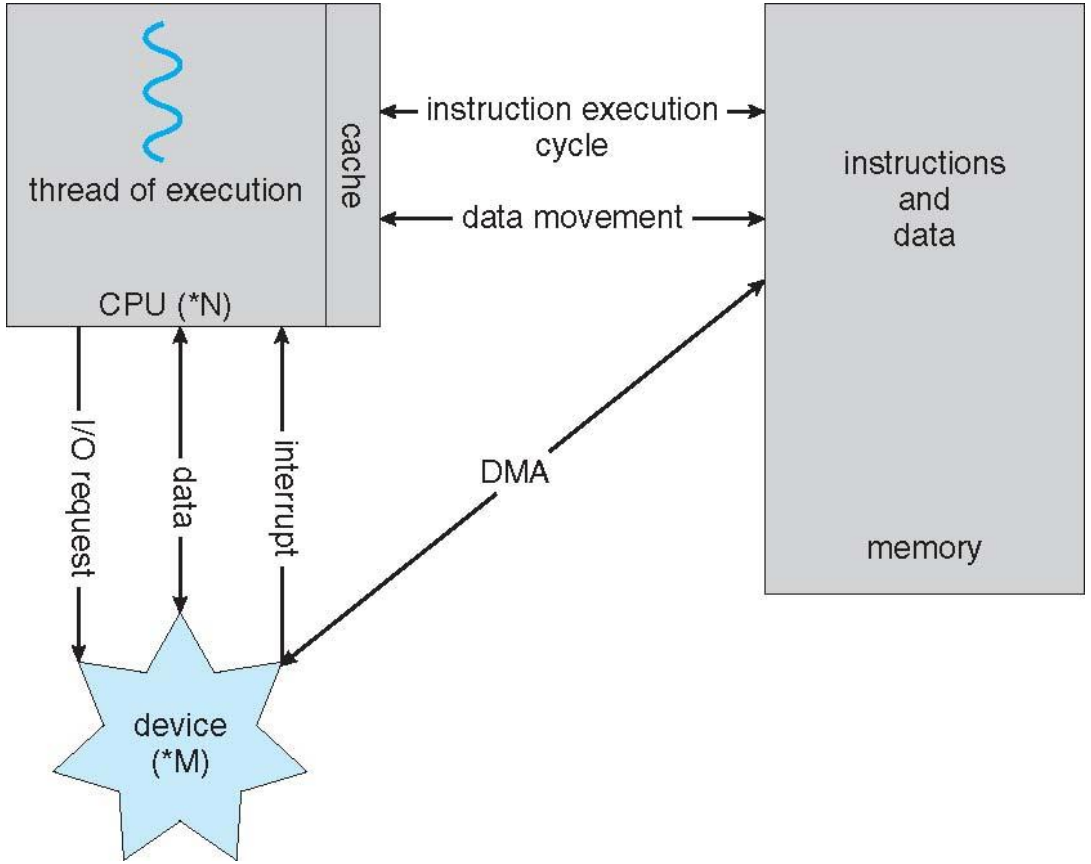
- Instruction Execution cycle

- Main difference lies in size, storage, cost and volatility

I/O Strucutre

- Computer – CPUs – multiple device controllers
- Device controller – one or more device attached
- SCSI controller – seven device
- Moves data between device and its local buffer.
- DMA

How a Modern Computer Works



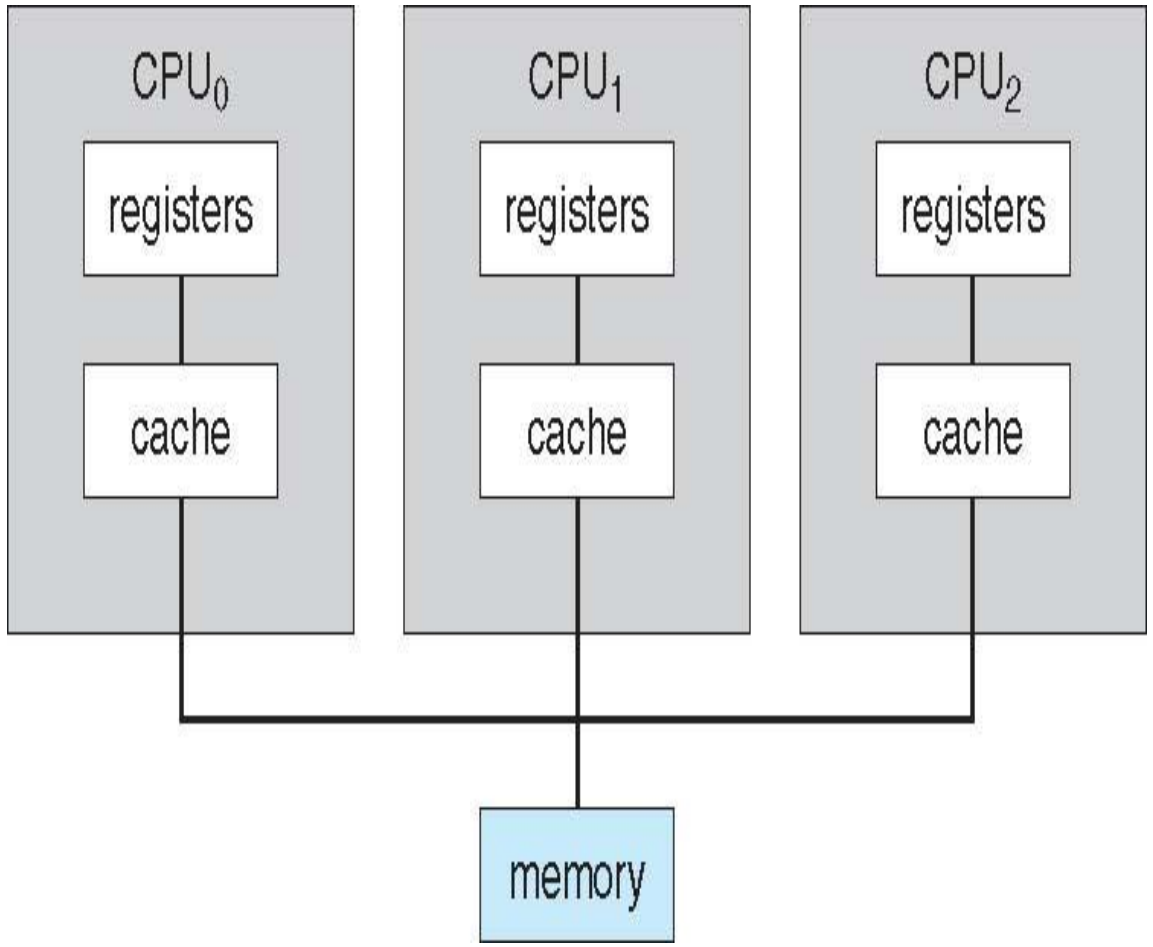
Computer-System Architecture

- Most systems use a single general-purpose processor
- Most systems have special-purpose processors as well
- Device-specific processors such as disk, keyboard and graphics controllers.
- Run a limited instruction set and do not run user processes.

Multi Processor System

- **Multiprocessors** systems growing in use and importance
  - Also known as **parallel systems**, **tightly-coupled systems**
  - Advantages include:
    1. **Increased throughput**
    2. **Economy of scale**
    3. **Increased reliability** – graceful degradation or fault tolerance
  - Two types:
    1. **Asymmetric Multiprocessing** – each processor is assigned a specific task.[boss and worker]
    2. **Symmetric Multiprocessing** – each processor performs all tasks

Symmetric Multiprocessing Architecture



Clustered System

- Gathers together multiple CPUs.
- Composed of two or more individual systems. - Loosely coupled
- Clustered computers share storage and are closely linked via a LAN.
- Used to provide high availability.[cluster software]
- High performance computing environment – run applications concurrently - parallelization

- Asymmetric Clustering - One machine is in hot-standby mode and the other is running applications.

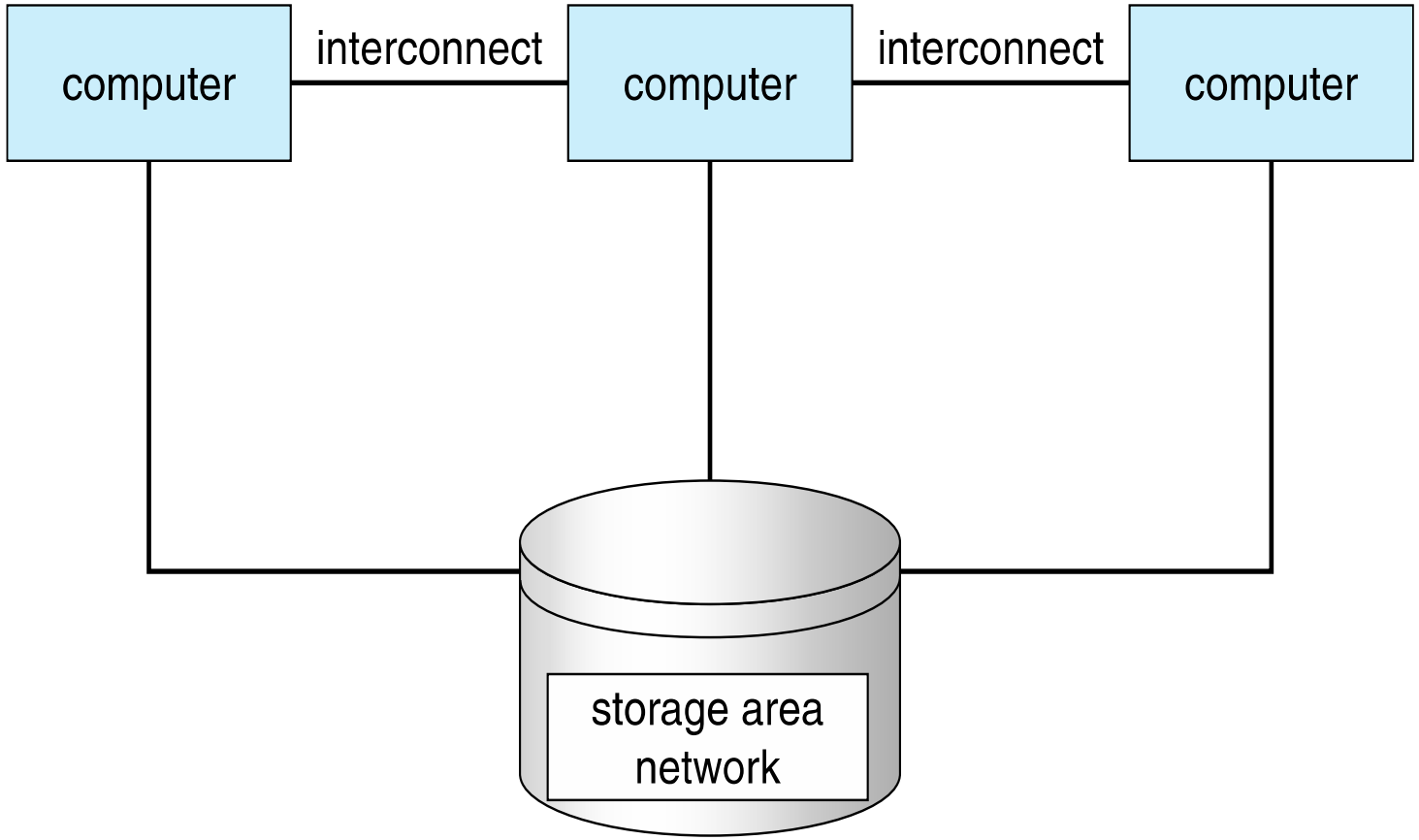
- Symmetric Clustering – two or more hosts are running applications and are monitoring each other.

- Parallelization – divide a program into separate components that run in parallel on individual computers in the cluster.

- Parallel clusters – allow multiple hosts to access the same data on a shared storage.

- Database cluster – dozens of hosts can share the same database thus increasing performance and reliability.

Clustered System



Operating system operations

- Operating systems are interrupt driven.
- For each type of interrupt, separate segments of code in the operating system determine what action should be taken.
- An interrupt service routine is provided to deal with the interrupt.
- Share hardware and software resources.
- A properly designed operating system must ensure that an incorrect program cannot cause other programs to execute incorrectly.

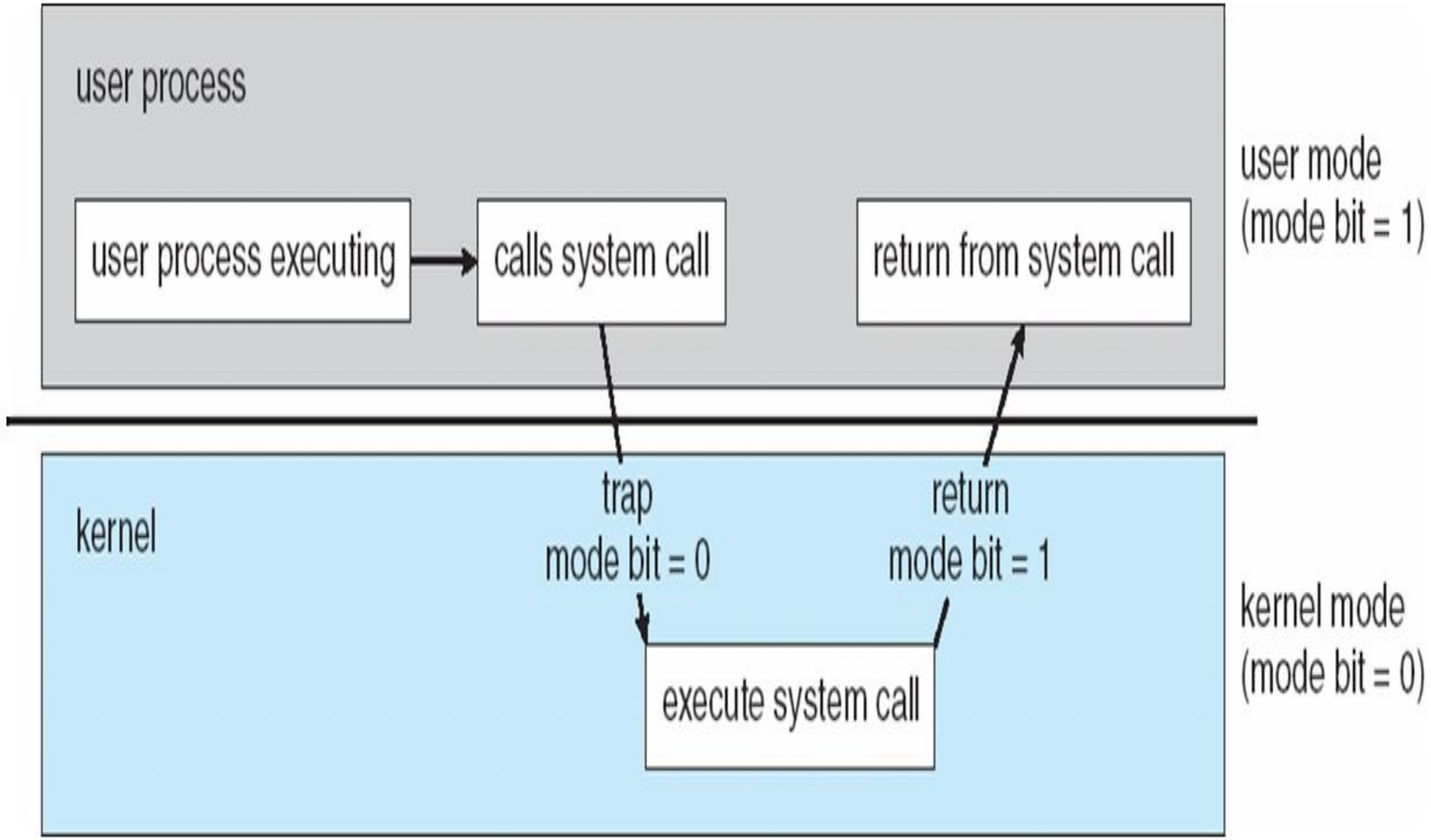
Dual-Mode and Multi-mode

- Differentiate operating system code and user defined code.
- Hardware support
- We need two separate modes of operation: user mode and kernel mode
- Mode bit – to indicate the current mode.

- Provides ability to distinguish when system is running user code or kernel code

- Dual mode of operation – protecting the operating system from errant users. – privileged instructions

- Example: switch to kernel mode
    - I/O control
    - Timer Management
    - Interrupt Management


user process
user process executing
calls system call
return from system call
user mode
(mode bit = 1)
kernel
trap
mode bit = 0
return
mode bit = 1
execute system call
kernel mode
(mode bit = 0)

- Can be extended beyond two modes
- CPUs that support virtualization have one more mode.
- Virtual Machine Manager
- Lack of hardware supported dual mode can cause serious shortcomings in an operating system

- Example:
- MS-DOS – no mode bit – no dual mode
- Multiple programs write over the same device
- Writing the data over the operating system

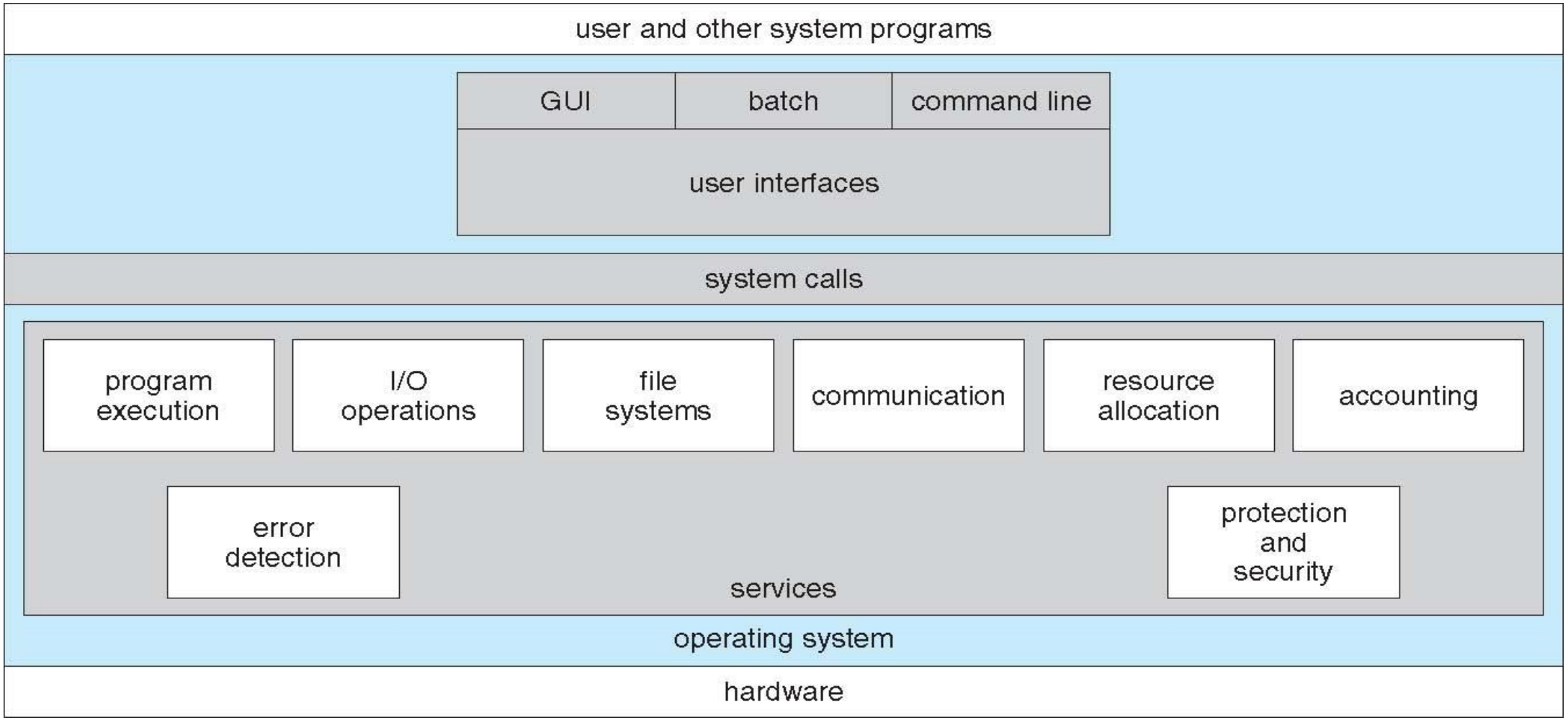
Timer

- Ensure that the operating system maintains control over the CPU.
- User program in infinite loop, fail to call system services, never return control to the operating system.
- To accomplish this, we need timer.
- A timer is set to interrupt the computer after a specified period.

- Timer – fixed or variable
- Clock and counter
- Everytime the clock ticks, the counter is decremented.
- When the counter reaches 0, interrupt occurs.
- When the interrupt occurs, the control is transferred to the operating system.
- Timer – to prevent a user program from running too long.

Operating System Services

- Operating system provides an environment for the execution of programs.
- It provides certain services to the programs and to the users of the programs.
- Services – differ
- To make the programming task easier.


user and other system programs
GUI
batch
command line
user interfaces
system calls
program execution
I/O operations
file systems
communication
resource allocation
accounting
error detection
protection and security
services
operating system
hardware

Operating system services

- **User Interface**
- Command Line Interface
- Batch Interface
- Graphical User Interface

- **Program Execution**
- System - Load program into memory and run
- End execution either normally or abnormally

- **I/O Operation**
- Running program may require I/O which may involve a file or an I/O device.
- Users cannot control I/O devices directly.
- Operating system must provide a means to do I/O.

- **File System Manipulation**
- Programs need to read and write files and directories.
- Create, delete, search, list file information, permission management.

- **Communication**
- One process needs to exchange information with another process.
- Same computer or different computer.
- Is implemented via shared memory or message passing

- **Error Detection**
- OS needs to detect and correct errors.
- Errors – CPU, memory hardware, I/O devices, User program.
- For each type of error, the operating system should take appropriate action.
- Halt , terminate the process, return an error code to the process.

- **Resource Allocation**
- For multiple users or multiple jobs – resource must be allocated.
- OS manages many types of resources.
- CPU cycles, memory, file storage, I/O devices.
- CPU scheduling.

- **Accounting**
- Keep track of which users use how much and what kinds of computer resources.
- Billed or statistics

- **Protection and Security**

- Protection involves ensuring that all access to system resources is controlled.

- Security starts with requiring each user to authenticate himself or herself to the system, usually by means of a password, to gain access to system resources.

System Calls

- Provide an interface to the services made available by an operating system.

- Written in C, C++ or assembly-language.

- Example: Write a simple program to read from one file and copy them to another file.

Example System Call Sequence

```
Acquire input file name
  Write prompt to screen
  Accept input
Acquire output file name
  Write prompt to screen
  Accept input
Open the input file
  if file doesn't exist, abort
Create output file
  if file exists, abort
Loop
  Read from input file
  Write to output file
Until read fails
Close output file
Write completion message to screen
Terminate normally
```

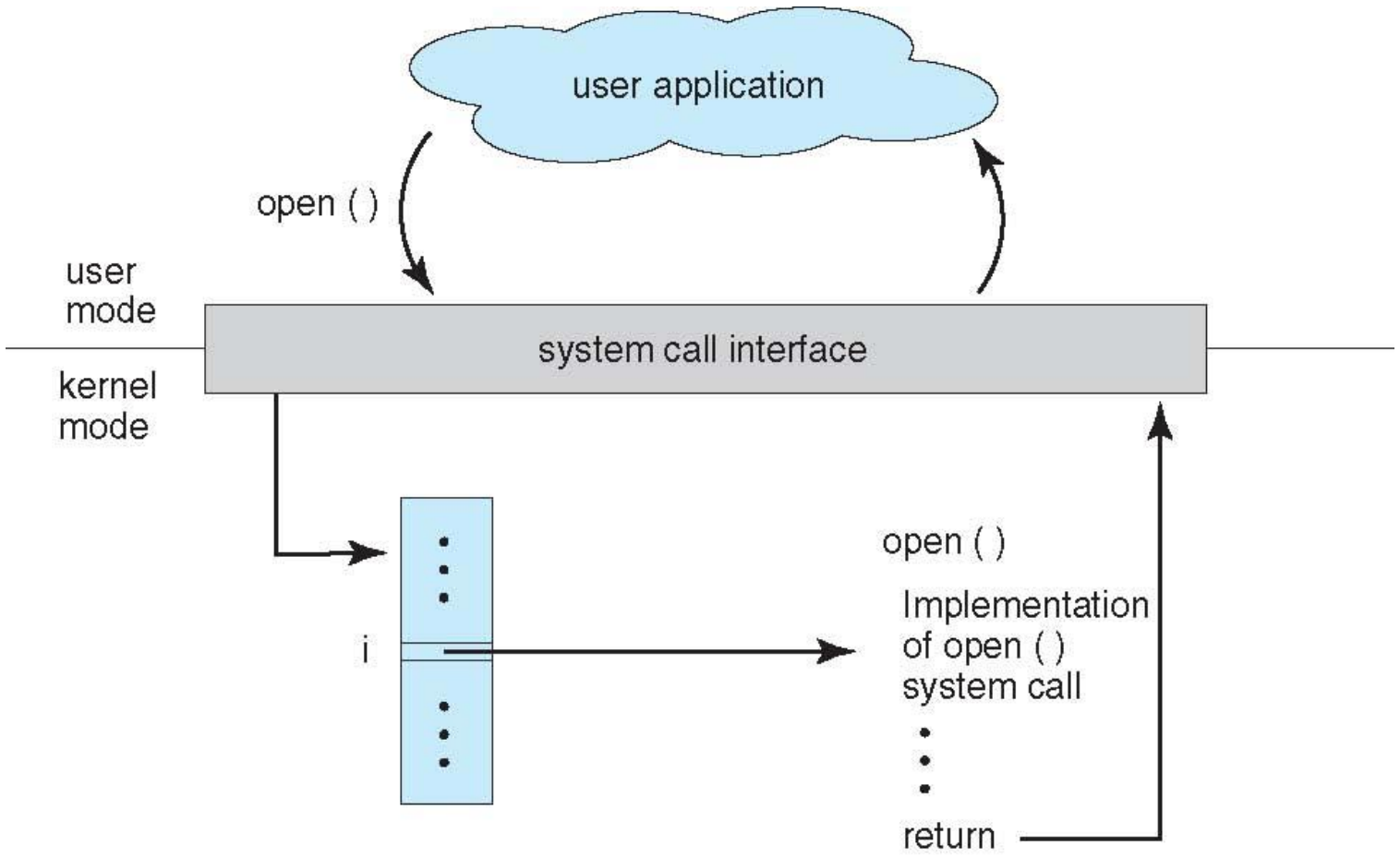
- Even simple programs may make heavy use of the operating system.

- Application developers design programs according to an API.

- Three common APIs available to application programmers:

- Windows API for Windows
- POSIX API for POSIX-based systems
- JAVA API for programs that run on JVM.

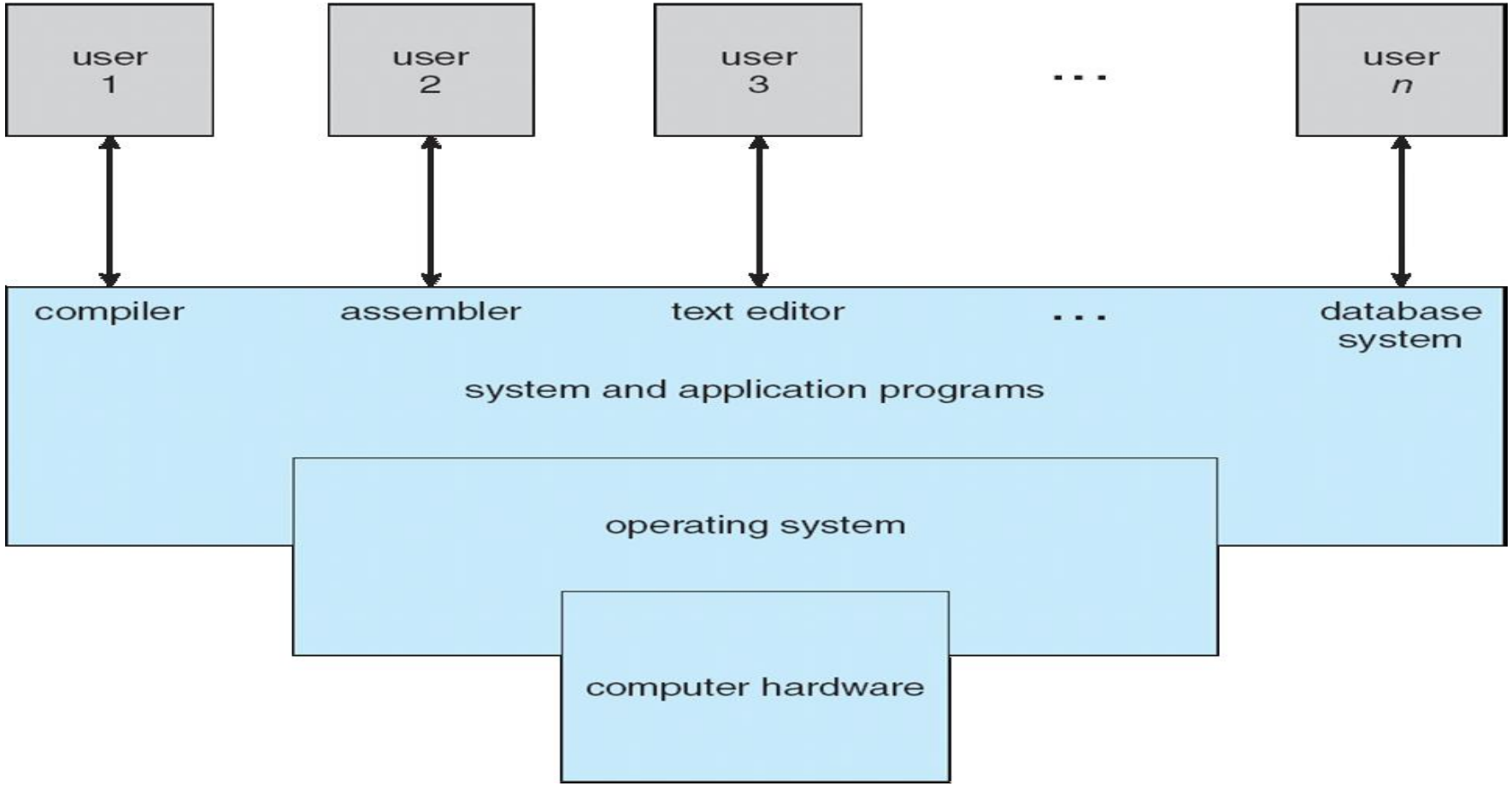
- Why application programmers prefer programming according to an API?
- Program Portability
- System calls are difficult to work than API
- Most of the details of the operating-system interface are hidden from the programmer by the API.


user application
open ( )
user
mode
system call interface
kernel
mode
i
open ( )
Implementation
of open ( )
system call
return

- Three methods are used to pass parameters to the operating system

- Pass parameters in Registers.
- Pass address of the block stored in register
- Push and pop the parameters from the stack

System Programs



- System programs are also known as system utilities.
- Provides a convenient environment for program development and execution.
- It can be simple such as user interface or it can be complex.

Categories of System Call

- **File Management** – create, delete, copy, rename, print, dump, list and manipulate files and directories.

- **Status Information** – ask for the date, time, amount of available memory, disk space, number of users, logging information, debugging information.
- It formats and prints the output to the terminal or other output devices.

- **File Modification** – Text editors are available to create and modify the file content.
- Special commands are available for searching the contents of files or some transformation of text.

- **Programming-Language Support** – Compilers, assemblers, debuggers and interpreters are available.

- **Program Loading and Execution** – Program is loaded into memory for execution.
- System provides absolute loaders, relocatable loaders, linkage editors and overlay loaders.

- **Communications** – Provide the mechanism for creating virtual connections among processes, users and computer system.

- **Background Services** – There are certain methods for launching certain system-program processes at boot time.
- Some processes terminate after finishing their tasks while others continue to run until the system is halted.
- These constantly running processes are called daemons or services or subsystems.

Operating System Structure

- For a system to function properly, it must be engineered carefully.
- Partition the task instead of monolithic.
- Carefully define the inputs, outputs and functions of these modules.

Simple Structure

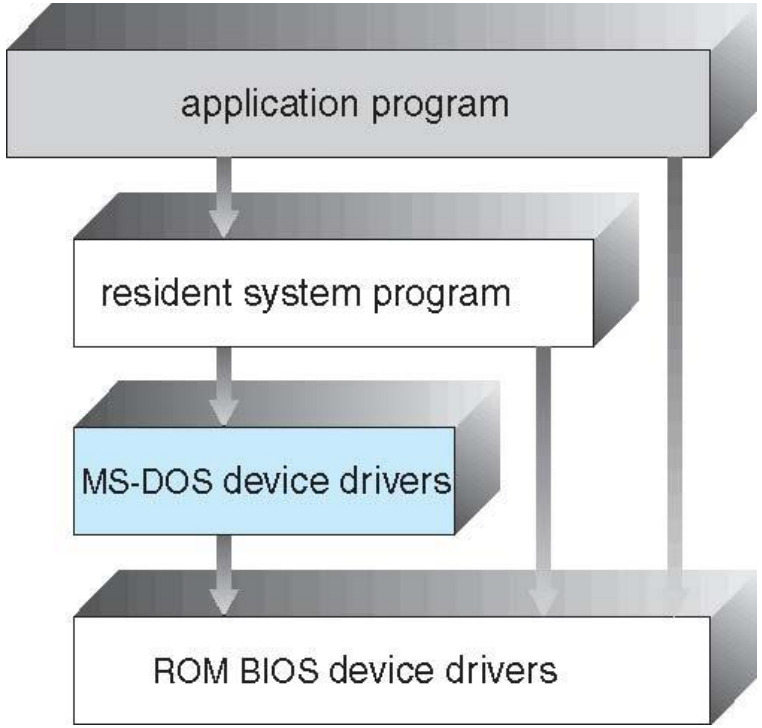
- Do not have well-defined structures.
- Small, simple and limited systems.
- Example: MS-DOS – written to provide more functionality in least space.

- The interfaces and the level of functionality are not well defined.

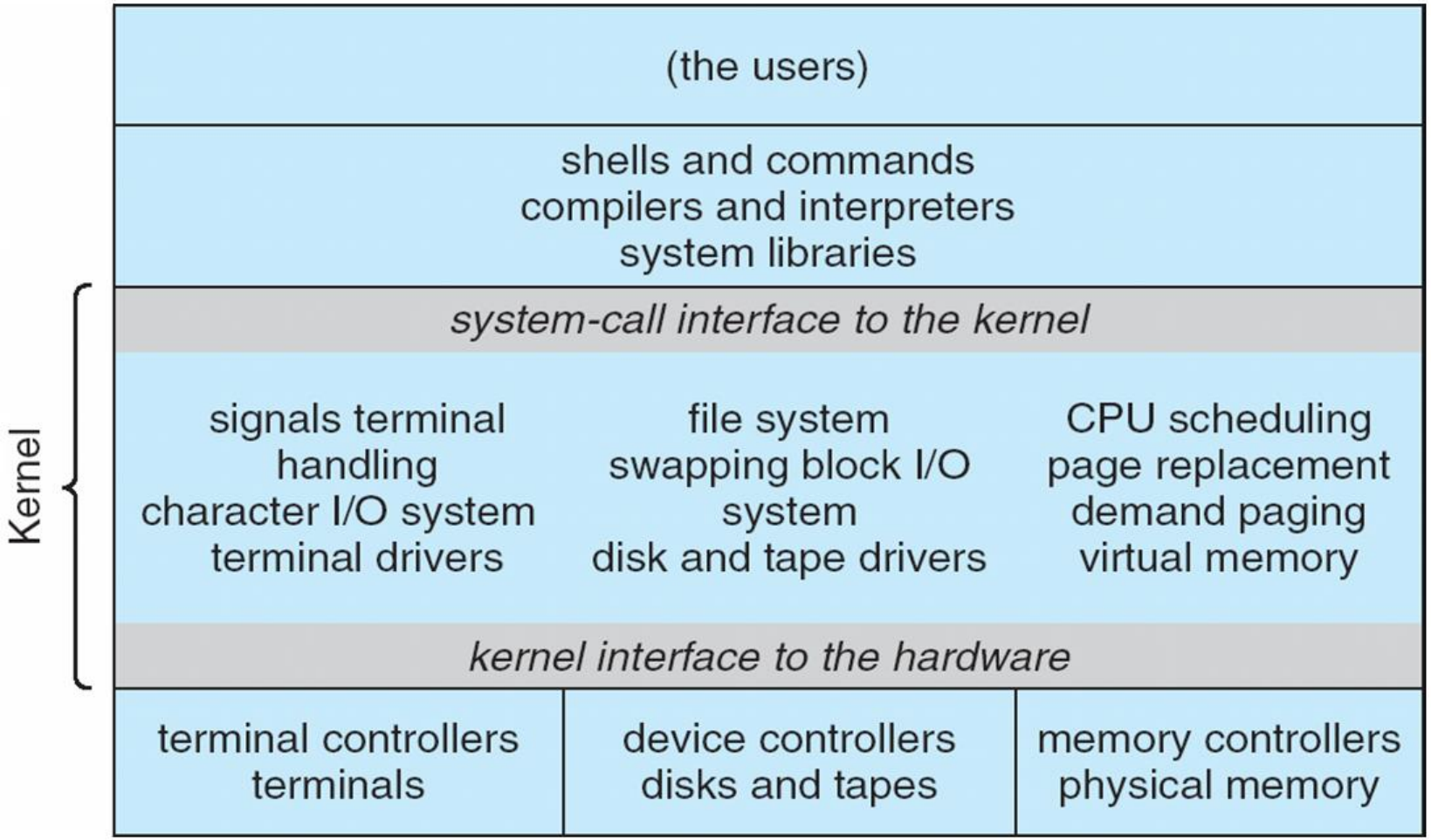
- Application programs are able to access Basic I/O routines directly.(write to display)

- This makes MS-DOS vulnerable.

- It provides limited hardware, No dual mode, no hardware protection.



- Another example: original Unix system.
- Limited hardware functionality.



- The structure of unix consists of two seperable parts:
- Kernel
- System Programs

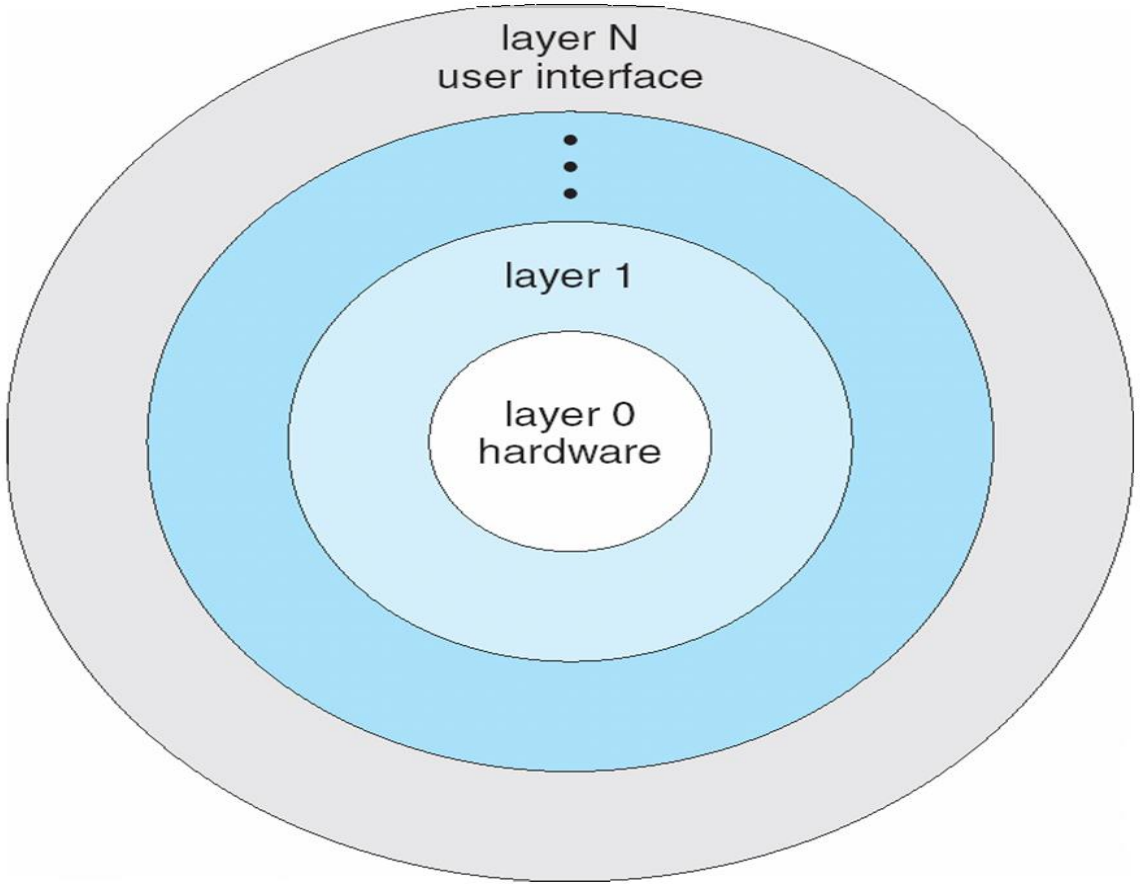
- Enormous amount of functionality is combined in one level – monolithic structure – difficult to implement and maintain.

Layered Approach

- Operating system can be broken into pieces – with hardware support.
- OS retain much greater control over the computer and over the applications.
- It makes the system more modular.
- In Layered approach, bottom layer is the hardware and the highest layer is the user interface.

Layered Operating System

- A typical operating system layer consists of set of data structures and routines that are invoked by the higher layers.



- Advantage:

Easy to construct and debug.

- Each layer uses functions and services of only lower level layers. – simplifies debugging and system verification.

- If error is found in any particular layer, the error must be on that layer, because the layers below it are already debugged.

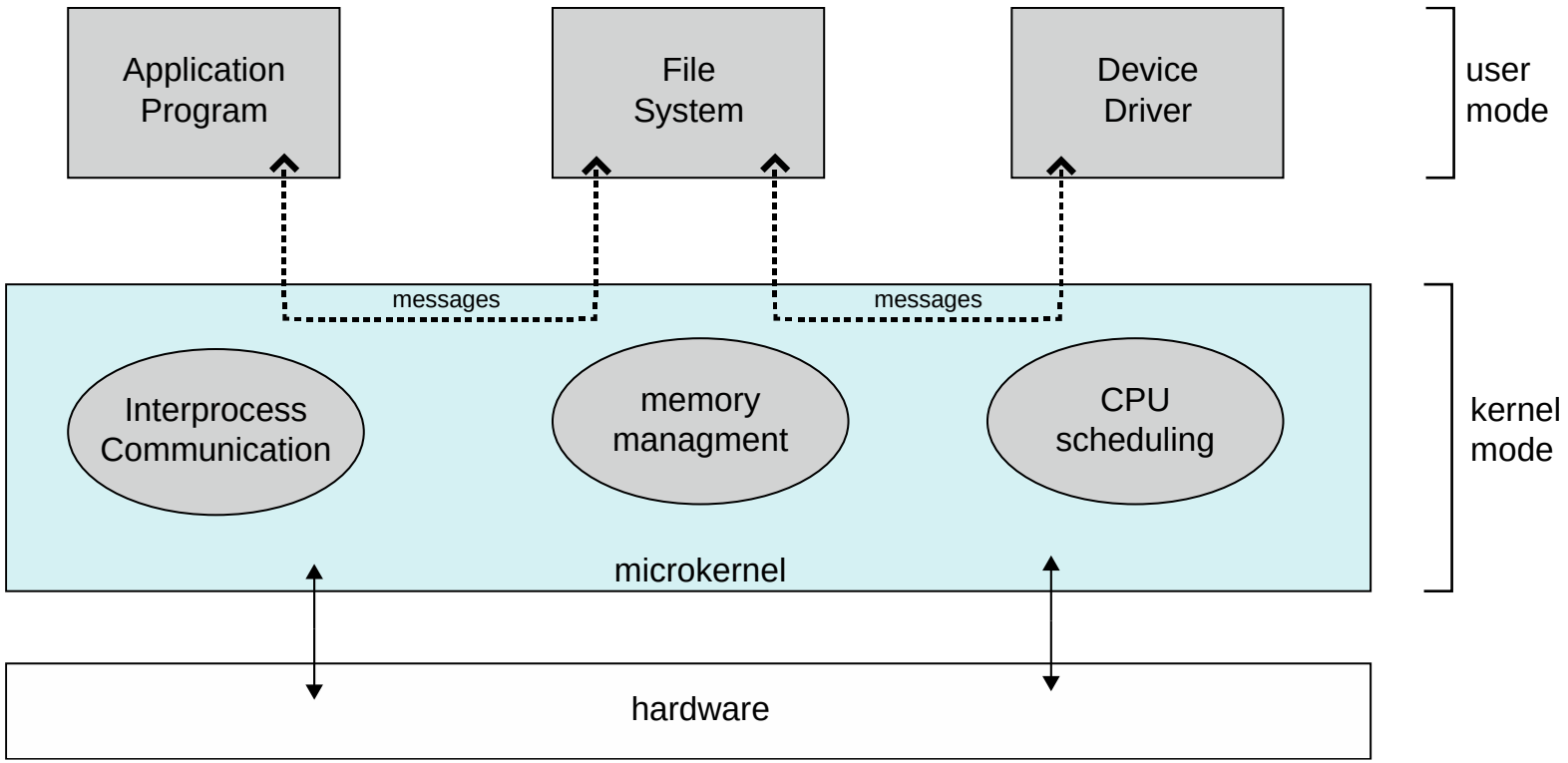
- Each layer hides the existence of certain data structures, operations and hardware from higher-level layers.

- Major difficulty is appropriately defining the various layers.
- Careful planning is necessary.
- Less efficient than other types – system call takes longer time.

Microkernels

- As unix expanded, Kernel become large and difficult to manage.
- Modularizing the kernel using micro kernel approach.
- Remove all non essential components from the kernel and implement them as system and user-level programs.

Architecture of a typical microkernel



- Main function of the microkernel is to provide communication between the client program and the various services that are running in the user space.
- Communication is provided through message passing.
- One advantage of microkernel is it is easy to extend the operating system further.
- Easy to port the operating system from one hardware to the other.
- Provides more security and reliability.

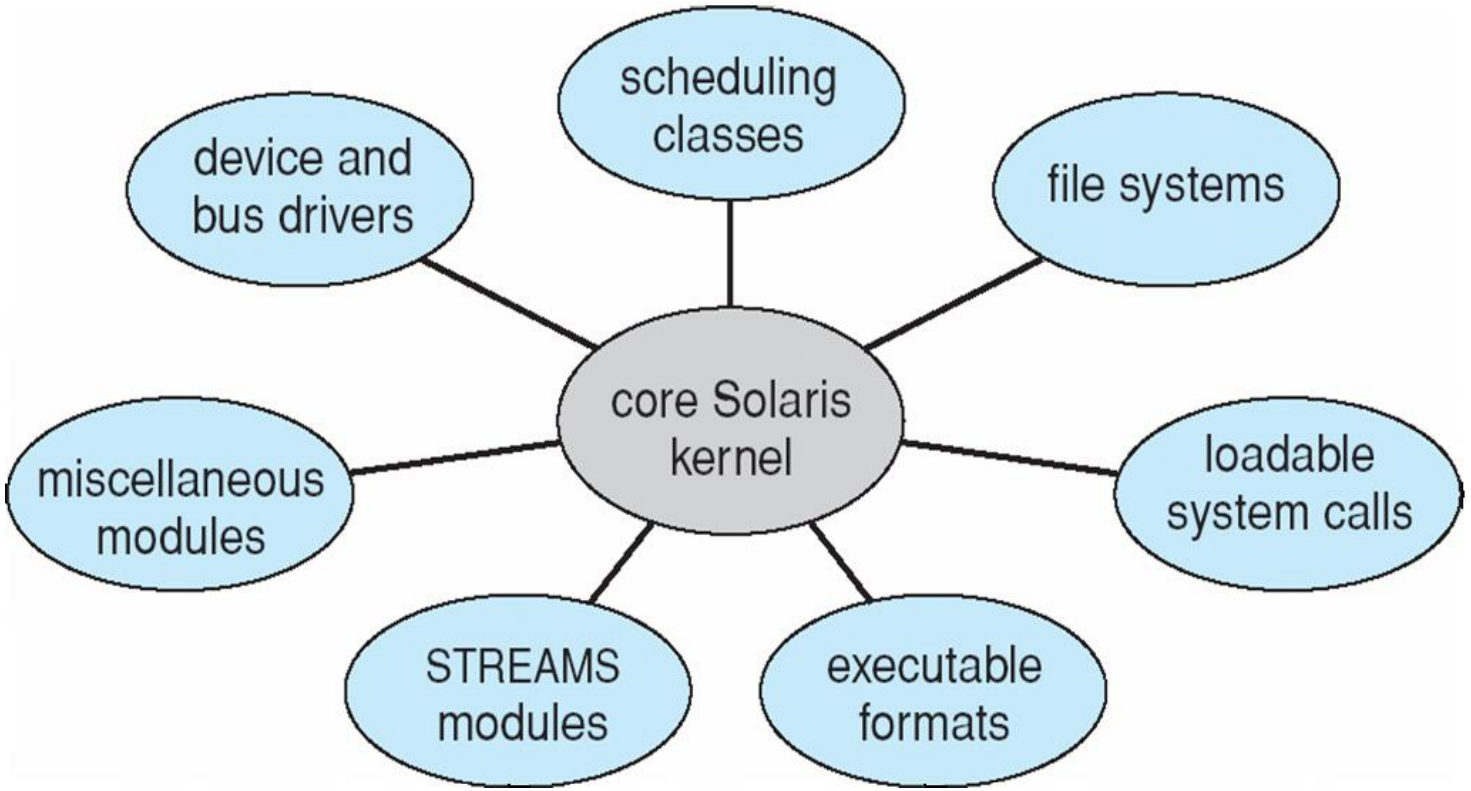
- Example:
- Tru64 UNIX
- Mac OS X
- QNX

- The performance of microkernels suffer due to increased system-function overhead.

Modules

- Using loadable kernel modules stands as the best methodology.
- Kernel has a set of core components and links in additional services in modules either at boot time or at run time.
- Example: Solaris, Linux, Windows.
- Build CPU scheduling algorithm in kernel and then add support for different file systems by way of loadable modules.
- It resembles a layered system but is more flexible than layered system.

- It is similar to microkernel approach but more efficient than microkernel approach as it do not involve message passing in order to communicate.



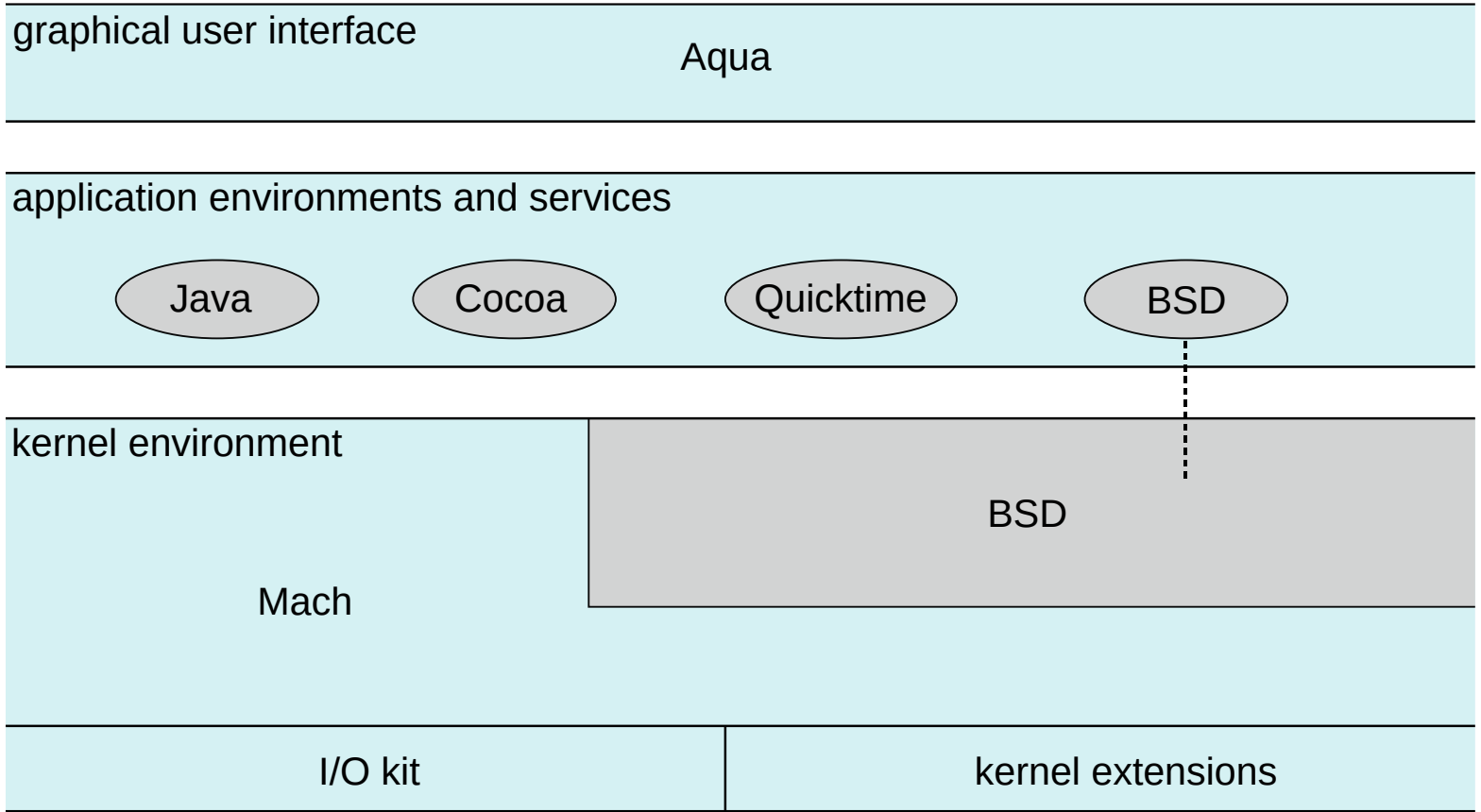
Hybrid Systems

- Combining different structures result in hybrid systems.
- Linux and Solaris – monolithic and modular
- Windows – largely monolithic and microkernel and provide support for dynamically loadable kernel modules.

Mac OS X

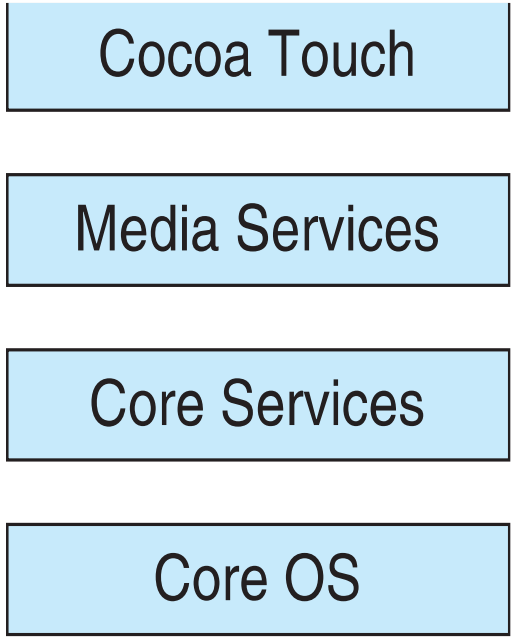
- Uses hybrid architecture.

- Layered system and Micro kernel.



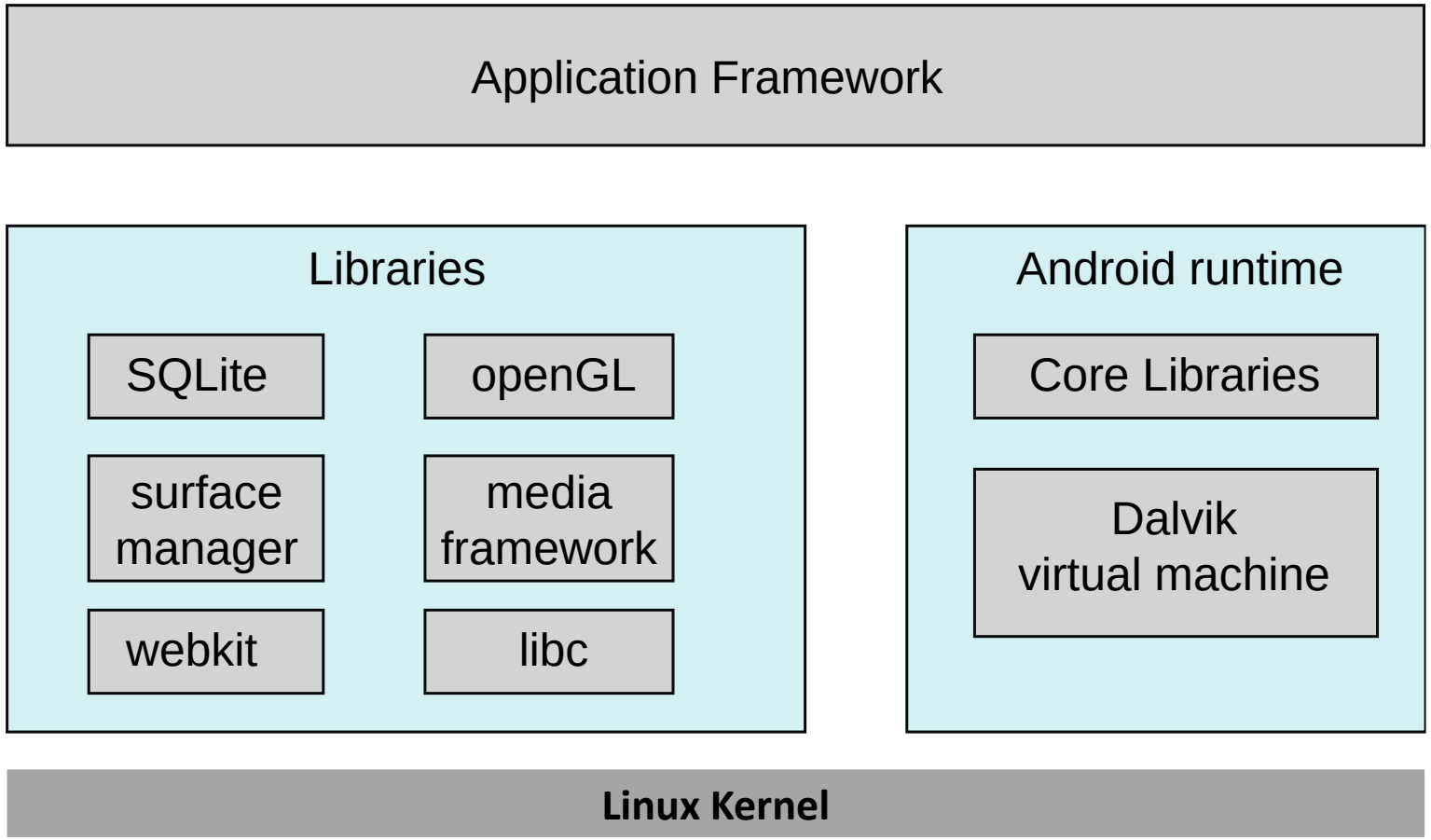
iOS

- Is structured on Mac OS X with additional functionality pertinent to mobile devices.



Android

- Open-sourced.



Operating system Generation

- Design, code and implement – OS – one machine at one site or class of machines at variety of sites.
- System must be configured or generated for each specific computer site – System generation (SYSGEN).
- OS is normally distributed on disk, CD-ROM or DVD-ROM or as an ISO image.

- SYSGEN program requires certain information concerning the specific configuration of the hardware system.

- What CPU is to be used?
- What options are installed?
- How will the boot disk be formatted?
- How many sections and partitions?
- How much memory is available?
- What devices are available?
- What parameter-values are to be used?
- What OS options are desired?

- After gathering the information, the system administrator modifies the source code of the operating system and then compile it completely and the system is generated.

System Boot

- Once the system is generated, it must be made available for use by the hardware.
- The hardware should know where the kernel is and how to load the kernel?
- The procedure of starting the computer by the loading the kernel is known as booting.
- This is done by the bootstrap program or bootstrap loader.
- It locates the kernel and loads into main memory and start its execution.

- Bootstrap program is in ROM – it needs no initialization, cannot be infected.

- Bootstrap program perform a variety of tasks.
- Run diagnostics to determine the state of the machine.
- Diagnostics pass – program can continue with booting.
- Can initialize all aspects – CPU registers, device controllers, main memory.
- Starts the operating system.

- Some systems – Cellular Phones, Tablets and game consoles – store operating system in ROM.
- Problem – changing the bootstrap code requires changing the ROM hardware chips.
- Solution – EPROM

- All forms of ROM – firmware.
- Executing code in firmware is slower than executing in RAM.
- Firmware – expensive.

- For large operating system, bootstrap is in firmware and OS is in disk.
- Bootstrap run the diagnostics and has a bit of code that reads a single block from disk into memory and execute that block.
- The program stored in this block loads the entire operating system into memory and begin its execution.
- Example: GRUB – open source bootstrap for LINUX.